



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

LEXICAL ANALYZER

Course: Compiladores

Group: 05

Professor: M.C. René Adrian Dávila Pérez

Student(s):

Ambriz Cano Diego Emilio

Casillas Juárez Camila

Ferrer Cordero José Manuel

González Casimiro Juan Carlos

Vidrio López Mario Alexis

Índice

Introducción	2
Expresiones regulares	2
Definición formal de cada token	3
Prioridad de reconocimiento	5
AFND	6
AFND. Lexical Analyzer	6
Conversión AFND $\rightarrow$ AFD	7
Método, con el ejemplo de clase	8
Conversión del AFND completo	8
AFD final	10
Consideraciones en el proceso de escaneo	12
Diagramas del proceso del lexer	14
Ciclo de escaneo	15
Pruebas del analizador léxico	16
T1. Caso correcto: declaración, cadenas y operador	16
T2. Comentarios, decimales y palabras reservadas	17
T3. Caso con error: constante de carácter mal formada	17
T4. Entrada para el analizador sintáctico	18
CFG	18
Left factoring	19
FIRST, FOLLOW, tabla LL(1) y traza.	19
Conclusiones	21

Introducción

Todo proceso de compilación tiene en esencia tres elementos; En primer lugar, el proceso de análisis léxico, sintáctico y semántico constituye el primer punto de contacto con el código fuente; el segundo elemento es una tabla de símbolos útil para la organización de todos los identificadores de un código fuente; el tercer es una fase de síntesis, en ella se genera un código intermedio para la ejecución final del programa.

Bajo la premisa anterior, este trabajo presenta el desarrollo del primer elemento de un compilador específicamente de la creación de un Analizador Léxico. Su objetivo fundamental consiste en transformar una secuencia continua de caracteres de entrada en una serie de unidades con significado sintáctico denominadas tokens, al mismo tiempo que descarta elementos no significativos como espacios en blanco y comentarios.

El resultado del proyecto es un Analizador Léxico capaz de generar una salida de tokens para validar una cadena de caracteres. Además, este analizador considera una lista de 45 tokens representativos para las palabras reservadas, identificadores, constantes de distintos tipos, literales de cadena, operadores y delimitadores de puntuación.

Expresiones regulares

A continuación se presenta la definición formal de la expresión regular por cada clase de token y por cada elemento descartable.

letra $\rightarrow [a-zA-Z]$
digito $\rightarrow [0-9]$
escape $\rightarrow \backslash$ (cualquier carácter excepto salto de línea)
identificador $\rightarrow (\text{letra} \mid _)(\text{letra} \mid \text{digito} \mid _)^*$
reservada $\rightarrow$ un lexema de identificador que está en la tabla de búsqueda
entero $\rightarrow \text{digito}^+$
decimal $\rightarrow \text{digito}^+ . \text{digito}^+$
caracter $\rightarrow '(\text{cualquier carácter excepto } ' , \backslash \text{ y salto de línea} \mid \text{escape})'$
booleano $\rightarrow \text{true} \mid \text{false}$ (se reconoce como identificador y se busca en la tabla)
constante $\rightarrow \text{entero} \mid \text{decimal} \mid \text{caracter} \mid \text{booleano}$
literal $\rightarrow "(\text{cualquier carácter excepto } " , \backslash \text{ y salto de línea} \mid \text{escape})^* "$
operador $\rightarrow + \mid - \mid * \mid / \mid \% \mid = \mid == \mid ! \mid != \mid < \mid <= \mid > \mid >= \mid \& \mid \&\& \mid \mid \mid \parallel$
puntuacion $\rightarrow (\mid) \mid \{ \mid \} \mid [\mid] \mid ; \mid ,$
especial $\rightarrow \# \mid .$
blanco $\rightarrow (\text{espacio} \mid \backslash t \mid \backslash r \mid \backslash n)^+$ se descarta
comentario $\rightarrow //(\text{cualquier carácter excepto salto de línea})^*$
 $\mid / * ([^*] \mid *^+ [^* /])^* *^+ /$ se descarta

Para los identificadores se consideró que el primer carácter puede ser una letra o guion bajo, mientras que los siguientes pueden incluir letras, dígitos o guion bajo. El decimal exige al menos un dígito después del punto, así que "5." no es una constante completa.

Los signos + y - no se consideran parte de las constantes numéricas, sino operadores independientes. De esta manera, una expresión como a-1 se reconoce como un identificador, un operador de resta y una constante entera.

El token del tipo "escape" "(símbolo \)" se aplica en cadenas y constantes, su uso permite al Lexer agregar comillas u otros caracteres distintos al salto de línea a cadenas de texto sin la interpretación de fin de cadena. Por ejemplo, en "Hello \"John\"", la comilla seguida del símbolo definido como escape no implica el final de la cadena.

Los tokens del tipo "Literal y Character" no admiten saltos de línea, en consecuencia, el analizador detecta un error en la línea donde empiezan la cadena si esta no se cierra correctamente. Las literales se refieren a las cadenas construidas entre comillas dobles, "...", mientras que los caracteres son constantes que contienen exactamente solo un elemento entre comillas simples ' '.

Los tokens de tipo boolean (true y false) son palabras, estas siguen el camino del identificador y se distinguen en la tabla de búsqueda; no necesitan estados propios. En los comentarios de bloque se permite cualquier contenido hasta encontrar la secuencia de cierre */. El patrón debe evitar que un asterisco aislado cierre el comentario de forma incorrecta.

El analizador lee todo el lexema hasta generar un token válido, aún si encuentra caracteres o identificadores catalogados a cierto token específico, se lee por completo. Al finalizar se compara con alguna de las palabras u operadores reservados.

Estas expresiones regulares sirven como base para la construcción de los autómatas. En la implementación no se van a usar directamente mediante una biblioteca de expresiones regulares, ya que serán representadas mediante el AFD que se va a construir más adelante.

Definición formal de cada token

A continuación se presenta una lista de 45 tokens agrupados en Palabra reservada, identificador, constante, literal, operador, puntuación y carácter especial.

#	TokenType	Clase (salida)	Lexema o patrón
1	INT	keyword	int
2	FLOAT	keyword	float
3	CHAR	keyword	char
4	DOUBLE	keyword	double
5	VOID	keyword	void
6	RETURN	keyword	return
7	IF	keyword	if
8	ELSE	keyword	else
9	WHILE	keyword	while

Tabla 1: Definición formal de cada token.

#	TokenType	Clase (salida)	Lexema o patrón
10	FOR	keyword	for
11	PRINT	keyword	print
12	ID	identifier	(letra _)(letra dígito _)*
13	INT_CONST	constant	uno o más dígitos
14	FLOAT_CONST	constant	dígitos, punto, dígitos
15	CHAR_CONST	constant	un carácter o un escape entre comillas simples
16	TRUE	constant	true
17	FALSE	constant	false
18	STRING	literal	cero o más caracteres o escapes entre comillas dobles
19	PLUS	operator	+
20	MINUS	operator	-
21	STAR	operator	*
22	SLASH	operator	/
23	PERCENT	operator	%
24	ASSIGN	operator	=
25	EQ	operator	==
26	NOT	operator	!
27	NEQ	operator	!=
28	LT	operator	<
29	LE	operator	<=
30	GT	operator	>
31	GE	operator	>=
32	AMP	operator	&
33	AND	operator	&&
34	BAR	operator	
35	OR	operator	
36	LPAREN	punctuation	(
37	RPAREN	punctuation	)
38	LBRACE	punctuation	{
39	RBRACE	punctuation	}
40	LBRACKET	punctuation	[
41	RBRACKET	punctuation	]
42	SEMI	punctuation	;
43	COMMA	punctuation	,
44	HASH	special	#
45	DOT	special	.

Tabla 1: Definición formal de cada token.

Prioridad de reconocimiento

La prioridad de los Tokens, cuando se reconoce más de un patrón a la entrada, se determinó a partir de las siguientes cinco reglas.

1. **Máximo bocado.** Toma el lexema más largo que termine en un estado final.
2. **Tabla antes que identificador.** Si el lexema termina en q_1 y está en la tabla de búsqueda, gana la tabla. Por ejemplo: `int` es palabra reservada y `true` es constante.
3. **Lo que no es token compite igual.** Espacios y comentarios se reconocen con las mismas reglas y después se descartan.
4. **Excepción del comentario sin cerrar.** Si la entrada se acaba dentro de un comentario de bloque (q_{10} o q_{11}), no se retrocede al / de q_8 : se reporta comentario sin cerrar.
5. **Sin estado final no hay token.** Si ningún prefijo llega a un estado final, se reporta un error léxico; no existe un token por defecto.

El máximo bocado es el primero porque toma la decisión en la mayoría de los casos y **nunca entra en conflicto con las otras reglas**. La tabla va después porque solo actúa cuando el lexema ya terminó en q_1 . Con estas reglas se resuelven los casos de ambigüedad considerados en el lenguaje definido para la práctica.

La regla cuatro se presenta en el AFD. Si `/* . . .` llega al final sin cerrarse, el último estado final visitado es q_8 ; sin la excepción, el escáner retrocedería y reportaría un operador `/` en vez del error real.

AFND

El siguiente autómata fue realizado en colaboración con los compañeros de clase durante una de las sesiones teóricas de la materia. Este sirve como base teórica para la construcción del autómata correspondiente a analizador léxico de este proyecto.

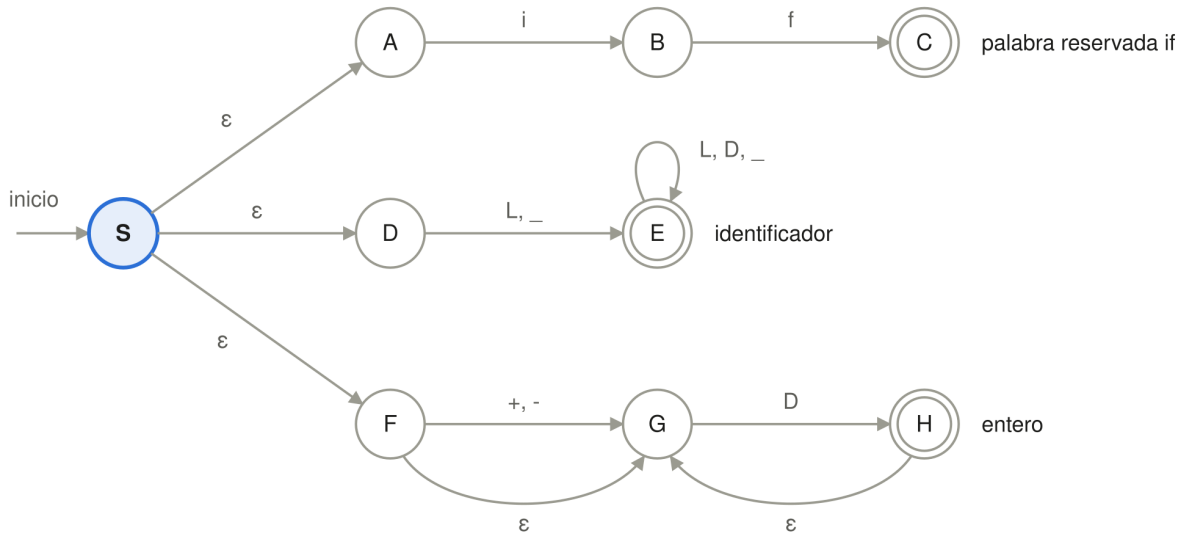


Figura 1: AFND realizado en sesiones teóricas.

$$S \xrightarrow{\varepsilon} A \quad S \xrightarrow{\varepsilon} D \quad S \xrightarrow{\varepsilon} F$$

$$A \xrightarrow{i} B \quad B \xrightarrow{f} C^*$$

$$D \xrightarrow{[a-zA-Z_]} E^* \quad E \xrightarrow{[a-zA-Z0-9_]} E$$

$$F \xrightarrow{+|-} G \quad F \xrightarrow{\varepsilon} G \quad G \xrightarrow{[0-9]} H^* \quad H \xrightarrow{\varepsilon} G$$

palabra reservada if

identificador

entero

(* = estado final)

AFND. Lexical Analyzer

Cada definición regular de la sección anterior se tradujo a un sub-autómata, es decir, un estado por cada punto del patrón donde se espera un tipo distinto de carácter, y un estado final donde el patrón puede terminar. Después, S se unió a los nueve estados iniciales del autómata de la sección anterior con movimientos ε .

Sub-autómata	Transiciones (origen $\rightarrow$ símbolo $\rightarrow$ destino)	Estados finales
Inicio	$S \xrightarrow{\varepsilon} I_0, N_0, T_0, K_0, C_0, O_0, P_0, X_0, W_0$	S : estado inicial
Identificador	$I_0 \xrightarrow{L} I_1^* \quad I_1 \xrightarrow{L D} I_1^*$	I_1 : identificador o reservada
Número	$N_0 \xrightarrow{D} N_1^* \quad N_1 \xrightarrow{D} N_1^* \quad N_1 \rightarrow N_2$ $N_2 \xrightarrow{D} N_3^* \quad N_3 \xrightarrow{D} N_3^*$	N_1 : entero N_3 : decimal
Cadena	$T_0 \xrightarrow{''} T_1 \quad T_1 \xrightarrow{c} T_1 \quad T_1 \xrightarrow{\backslash} T_2$ $T_2 \xrightarrow{a} T_1 \quad T_1 \xrightarrow{''} T_3^*$	T_3 : literal (cadena)
Carácter	$K_0 \xrightarrow{' } K_1 \quad K_1 \xrightarrow{k} K_2 \quad K_1 \xrightarrow{\backslash} K_3$ $K_3 \xrightarrow{a} K_2 \quad K_2 \xrightarrow{' } K_4^*$	K_4 : constante de carácter
Comentarios	$C_0 \xrightarrow{/} C_1 \quad C_1 \xrightarrow{/} C_2^* \quad C_2 \xrightarrow{a} C_2^*$ $C_1 \xrightarrow{*} C_3 \quad C_3 \xrightarrow{m} C_3 \quad C_3 \xrightarrow{*} C_4$ $C_4 \xrightarrow{*} C_4 \quad C_4 \xrightarrow{n} C_3 \quad C_4 \xrightarrow{/} C_5^*$	C_2 : comentario de línea C_5 : comentario de bloque (ambos se descartan)
Operadores	$O_0 \xrightarrow{+ - * \% /} O_1^* \quad O_0 \xrightarrow{! < >} O_2^* \quad O_2 \xrightarrow{=} O_3^*$ $O_0 \xrightarrow{\&} O_4^* \quad O_4 \xrightarrow{\&} O_3^* \quad O_0 \xrightarrow{ } O_5^*$ $O_5 \xrightarrow{ } O_3^*$	O_1, O_2, O_4, O_5 : un carácter O_3 : dos caracteres
Puntuación	$P_0 \xrightarrow{\text{PUNCT}} P_1^*$	P_1 : puntuación
Especial	$X_0 \xrightarrow{\#} X_1^*$	X_1 : carácter especial
Blanco	$W_0 \xrightarrow{\text{WS, NL}} W_1^* \quad W_1 \xrightarrow{\text{WS, NL}} W_1^*$	W_1 : blanco (se descarta)

Tabla 2. Transiciones y estados finales por cada sub-autómata del Lexer.

Símbolos

L = letra o $_$

c = cualquier carácter excepto $'$, $\backslash$ y NL

m = cualquier carácter excepto $*$

D = dígito

k = cualquier carácter excepto $'$, $\backslash$ y NL

n = cualquier carácter excepto $*$ y $/$

PUNCT = $() \{ \} [] ; ,$

a = cualquier carácter excepto NL

$[]$ = cualquiera de los símbolos indica

Único no determinismo: S , que con ε abre los nueve sub-autómatas a la vez, y el carácter $/$, que lleva tanto a O_1 (operador) como a C_1 (comentario). Cada sub-autómata, por separado, ya es determinista.

Conversión AFND $\rightarrow$ AFD

Un AFD tiene exactamente un movimiento por estado y símbolo, así que se ejecuta en una sola pasada con una consulta a la tabla por carácter. El AFD resultante puede implementarse mediante una tabla de transiciones, permitiendo determinar el siguiente estado a partir del estado actual y la clase del carácter leído.

Método, con el ejemplo de clase

Se usan dos operaciones. ε -cerradura(T) son los estados a los que se llega desde T solo con ε ; mover(T, a) son los estados a los que se llega desde T con el símbolo a . Cada estado del AFD es un conjunto de estados del AFND.

1. $Q_0 = \varepsilon$ -cerradura(S). Desde S , ε lleva a A , D y F ; desde F , ε lleva a G . $Q_0 = \{S, A, D, F, G\}$.
2. Desde Q_0 con i : A pasa a B y D pasa a E . Resultado $\{B, E\} = Q_1$.
3. Desde Q_0 con f , otra letra o $_$: solo D se mueve, a E . Resultado $\{E\} = Q_2$.
4. Desde Q_0 con un dígito: G pasa a H , y por ε H regresa a G . Resultado $\{G, H\} = Q_3$.
5. Desde Q_0 con $+$ o $-$: F pasa a G . Resultado $\{G\} = Q_4$.
6. Desde Q_1 con f : B pasa a C y E se queda en E . Resultado $\{C, E\} = Q_5$.
7. Desde Q_1 con otra letra o un dígito: solo E se mueve. Resultado $\{E\} = Q_2$.
8. Q_2 se cicla en sí mismo; Q_3 y Q_4 con dígito van a Q_3 ; Q_5 con letra o dígito va a Q_2 (ifx es identificador).
9. Un estado del AFD es final si contiene algún estado final del AFND.

Estado AFD	Estados AFND	i	f	otra letra, _	dígito	+ o -	¿Final?
Q_0	$\{S, A, D, F, G\}$	Q_1	Q_2	Q_2	Q_3	Q_4	no
Q_1	$\{B, E\}$	Q_2	Q_5	Q_2	Q_2	—	identificador
Q_2	$\{E\}$	Q_2	Q_2	Q_2	Q_2	—	identificador
Q_3	$\{G, H\}$	—	—	—	Q_3	—	entero
Q_4	$\{G\}$	—	—	—	Q_3	—	no
Q_5	$\{C, E\}$	Q_2	Q_2	Q_2	Q_2	—	if (prioridad sobre identificador)

Tabla 3. Transiciones de estados para el ejemplo de conversión.

Q_5 contiene estados finales de dos clases, C (if) y E (identificador), esto se resuelve con la regla 2 “Tabla antes que identificador”. El resultado es el mismo que reconocer todo como identificador y luego consultar una tabla.

Conversión del AFND completo

ε -cerradura(S) da q_0 , y cada símbolo de entrada lleva a un conjunto de estados del AFND que se vuelve un estado del AFD:

Estado AFD	Conjunto del AFND	Estado AFD	Conjunto del AFND
q_0	$\{S, I_0, N_0, T_0, K_0, C_0, O_0, P_0, X_0, W_0\}$	q_{13}	$\{O_3\}$
q_1	$\{I_1\}$	q_{14}	$\{O_4\}$
q_2	$\{N_1\}$	q_{15}	$\{O_5\}$
q_3	$\{N_2\}$	q_{16}	$\{P_1\}$
q_4	$\{N_3\}$	q_{17}	$\{X_1\}$
q_5	$\{T_1\}$	q_{18}	$\{C_5\}$
q_6	$\{T_3\}$	q_{19}	$\{W_1\}$
q_7	$\{O_1\}$	q_{20}	$\{T_2\}$
q_8	$\{O_1, C_1\}$	q_{21}	$\{K_1\}$
q_9	$\{C_2\}$	q_{22}	$\{K_3\}$
q_{10}	$\{C_3\}$	q_{23}	$\{K_2\}$
q_{11}	$\{C_4\}$	q_{24}	$\{K_4\}$
q_{12}	$\{O_2\}$		

Tabla 4. Correspondencia de estados entre el AFND completo y el AFD resultante.

Solo q_0 y q_8 agrupan más de un estado del AFND, porque ahí estaba el único no determinismo. Los demás coinciden uno a uno, por eso el AFD se puede dibujar como una fila por clase.

AFD final

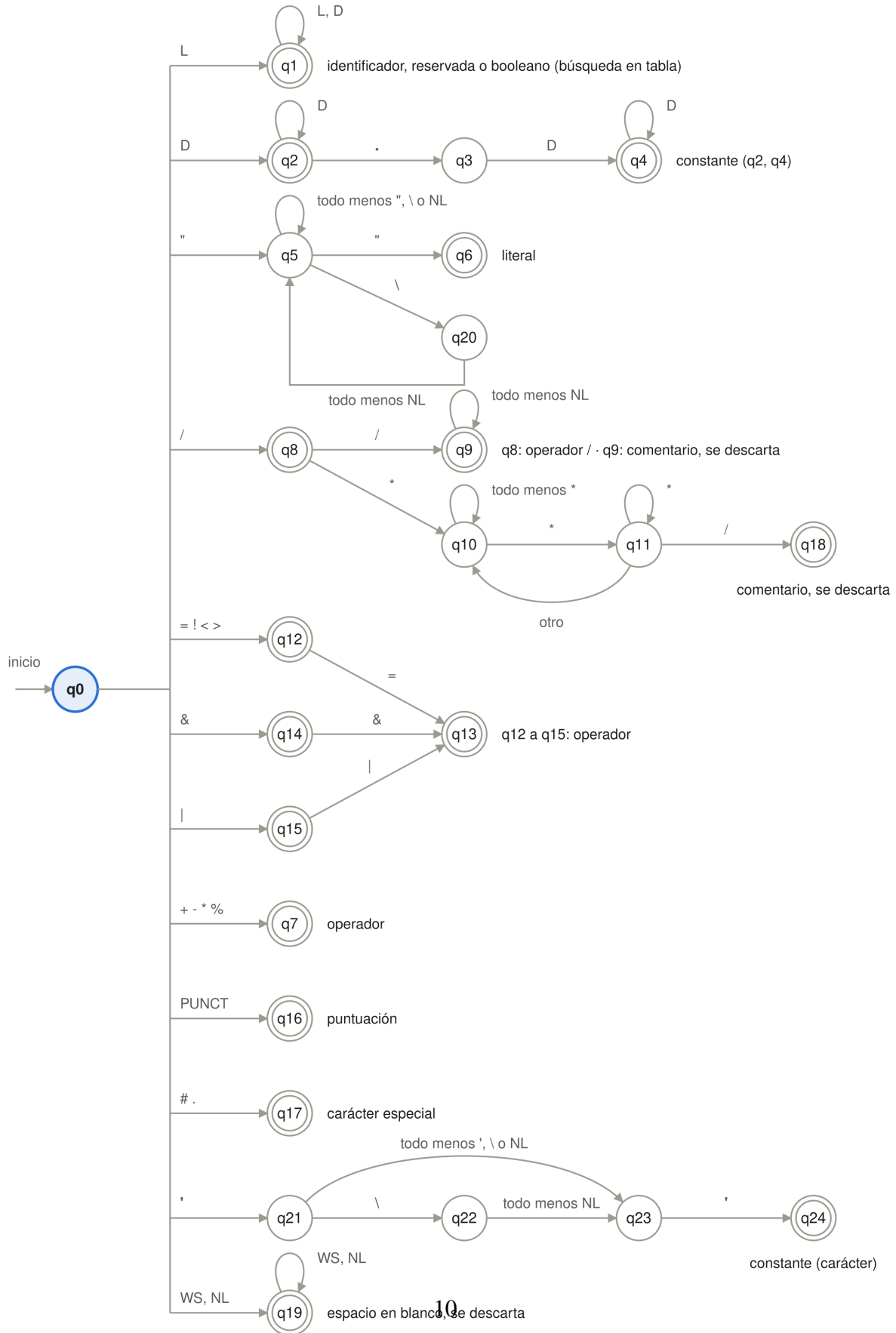


Figura 2: AFD del lexer · 25 estados, 16 finales

Cada clase de token tiene su propia fila, así que el estado final donde se detiene el escaneo fija la clase del lexema.

Algunos estados correspondientes a operadores presentan el mismo comportamiento respecto a las clases de entrada, por lo que pueden representarse mediante un mismo estado. Por ejemplo, los operadores =, !, < y > comparten un estado intermedio, ya que pueden finalizar individualmente o continuar con = para formar un operador de dos caracteres. Por eso = ! < > comparten q_{12} y todos los operadores de dos caracteres terminan en q_{13} . q_{14} y q_{15} no se fusionan, porque q_{14} avanza con & y q_{15} con |.

Dos caracteres van en la misma columna si se comportan igual en todos los estados. Así, las 128 columnas de ASCII se reducen a 18. Esta agrupación evita definir una columna independiente para cada carácter ASCII. Por ejemplo, todas las letras se comportan de la misma manera en las transiciones asociadas a identificadores, por lo que se agrupan en la clase L.

Clase	Caracteres	Clase	Caracteres
L	a-z, A-Z, _	AMP	&
D	0-9	BAR	
DOT	.	ARITH	+ - %
QUOTE	"	PUNCT	() { } [] ; ,
APOS	'	HASH	#
BSLASH	\	WS	espacio, tabulador, retorno de carro
SLASH	/	NL	salto de línea
STAR	*	OTHER	cualquier otro carácter
EQ	=		
REL	! < >		

Tabla 5. Agrupación de los caracteres ASCII en clases de equivalencia para el AFD.

Tabla de transiciones $\delta(\text{estado}, \text{clase})$; un guion significa que no hay movimiento:

STATE	L	D	DOT	QUOTE	APOS	BSLASH	SLASH	STAR	EQ	REL	AMP	BAR	ARITH	PUNCT	HASH	WS	NL	OTHER
q_0	q_1	q_2	q_{17}	q_5	q_{21}	—	q_8	q_7	q_{12}	q_{12}	q_{14}	q_{15}	q_7	q_{16}	q_{17}	q_{19}	q_{19}	—
q_1	q_1	q_1	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
q_2	—	q_2	q_3	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
q_3	—	q_4	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
q_4	—	q_4	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
q_5	q_5	q_5	q_5	q_6	q_5	q_{20}	q_5	q_5	q_5	q_5	q_5	q_5	q_5	q_5	q_5	q_5	—	q_5
q_6	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
q_7	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
q_8	—	—	—	—	—	—	q_9	q_{10}	—	—	—	—	—	—	—	—	—	—
q_9	q_9	q_9	q_9	q_9	q_9	q_9	q_9	q_9	q_9	q_9	q_9	q_9	q_9	q_9	q_9	q_9	—	q_9
q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{11}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}
q_{11}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{18}	q_{11}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}
q_{12}	—	—	—	—	—	—	—	—	q_{13}	—	—	—	—	—	—	—	—	—
q_{13}	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
q_{14}	—	—	—	—	—	—	—	—	—	—	q_{13}	—	—	—	—	—	—	—
q_{15}	—	—	—	—	—	—	—	—	—	—	—	q_{13}	—	—	—	—	—	—
q_{16}	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
q_{17}	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
q_{18}	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
q_{19}	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	q_{19}	q_{19}
q_{20}	q_5	q_5	q_5	q_5	q_5	q_5	q_5	q_5	q_5	q_5	q_5	q_5	q_5	q_5	q_5	q_5	—	q_5
q_{21}	q_{23}	q_{23}	q_{23}	q_{23}	—	q_{22}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	—	q_{23}
q_{22}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	—	q_{23}
q_{23}	—	—	—	—	q_{24}	—	—	—	—	—	—	—	—	—	—	—	—	—
q_{24}	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—

Tabla 6. Transiciones del AFD (25 estados $\times$ 18 clases).

Estados finales (q_0 , q_3 , q_5 , q_{10} , q_{11} y q_{20} a q_{23} no son finales):

Estado final	Lexemas que termina	Regresa
q_1	palabras	el TokenType de la tabla de búsqueda; si no está, ID
q_2	10	INT_CONST
q_4	3.14	FLOAT_CONST
q_{24}	'a', '\n'	CHAR_CONST
q_6	"..."	STRING
$q_7, q_8, q_{12}, q_{13}, q_{14}, q_{15}$	+ - *% / = ! < > == != <= >= & &&	el operador según el lexema (PLUS, EQ, AND...)
q_{16}	() { } [] ; ,	la puntuación según el lexema (LPAREN, SEMI...)
q_{17}	# .	HASH o DOT
q_9, q_{18}	// ..., /* ... */	comentario, se descarta
q_{19}	espacios, tabuladores, saltos de línea	espacio en blanco, se descarta

Tabla 7. Mapeo de estados finales del AFD a sus respectivos tipos de token y lexemas.

Consideraciones en el proceso de escaneo

Un error léxico ocurre cuando el escaneo se detiene sin haber pasado por un estado final. Para el AFD construido, los principales casos en los que el escaneo puede detenerse sin obtener un token válido son en los siguientes estados:

Estado de confusión	Qué significa	Mensaje	Recuperación
q_0	carácter que no inicia ningún token, como @ o \	Lexical error 1:5: unexpected character '@'	saltar ese carácter
q_5 o q_{20}	cadena sin cerrar en la misma línea	Lexical error 2:8: unterminated string literal	saltar al final de la línea
q_{21} , q_{22} o q_{23}	constante de carácter mal formada: '', 'ab' o sin cerrar	Lexical error 3:10: malformed character constant	saltar hasta la siguiente ' de la línea, o al final de la línea
q_{10} o q_{11} , al final de la entrada	comentario de bloque sin cerrar (regla 4)	Lexical error 4:1: unterminated comment	dejar de escanear

Tabla 8. Manejo de errores léxicos.

En q_3 no hay error: 5. siempre tiene antes el estado final q_2 , así que se emite 5 y se vuelve a escanear desde .. La recuperación del carácter salta hasta la siguiente comilla para que 'ab' no se convierta en un identificador falso más otro error.

Diagramas del proceso del lexer

El diagrama de flujo muestra cómo los errores y los elementos descartados regresan al inicio del ciclo sin agregar tokens. La ejecución del Lexer se realiza en 5 etapas mostradas en el siguiente gráfico.

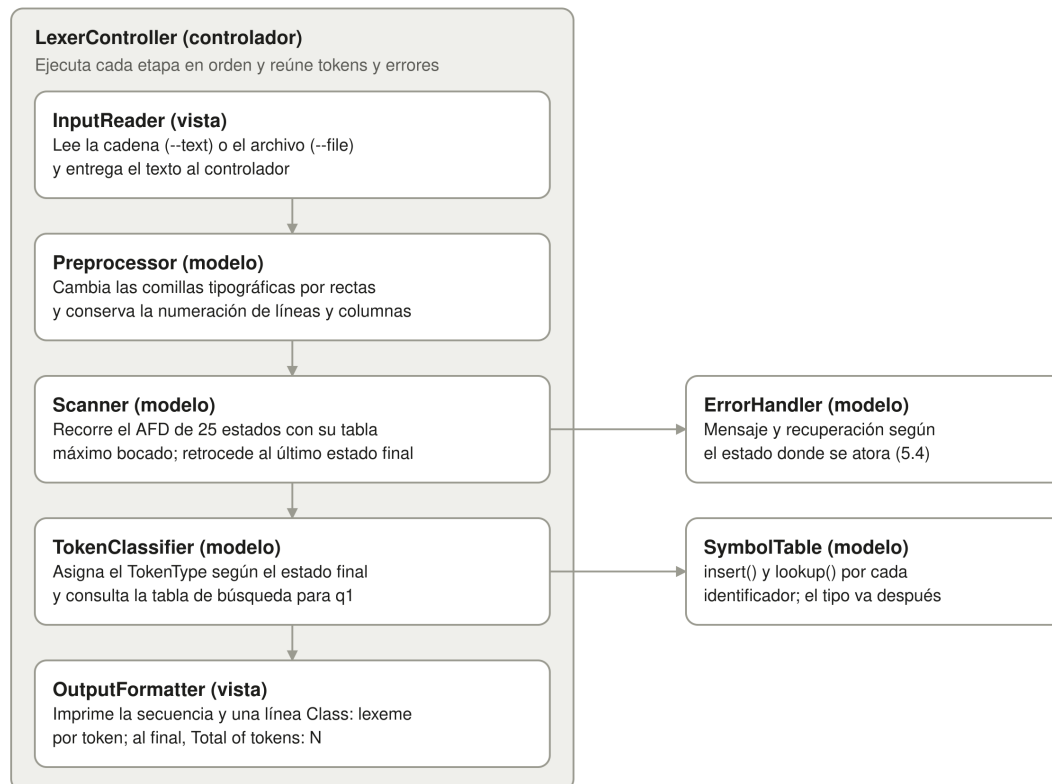


Figura 3: Etapas del lexer

Ciclo de escaneo

En la siguiente imagen se muestra un diagrama de flujo del proceso scan del Lexer.

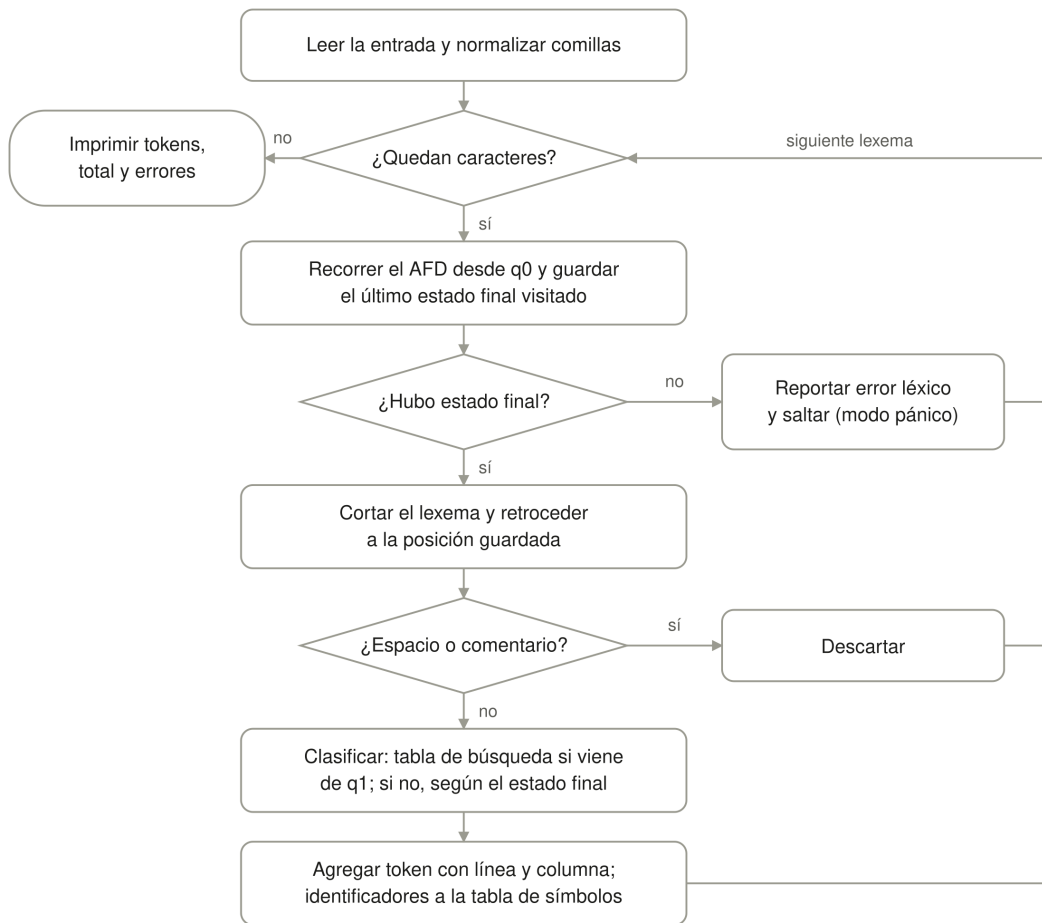


Figura 4: Diagrama de flujo del escaneo

Cada vuelta del ciclo consume un lexema; los errores y los elementos descartados regresan al inicio sin agregar tokens.

La construcción de este diagrama se obtuvo a partir del pseudocódigo; if es un rombo de decisión, así mismo, cada grupo de instrucciones a una caja.

```
while pos < len(text):
    state = q0; i = pos; lastFinal = none; lastEnd = pos
    while i < len(text):
        next = delta[state][classOf(text[i])]
        if next == none: break
        state = next; i = i + 1
        if state in FINAL: lastFinal = state; lastEnd = i
    if i == len(text) and state in {q10, q11}:
        errors.report(line, col, "unterminated comment"); break
```

```

if lastFinal == none:
    errors.report(line, col, state); pos = recover(pos, state); continue
lexeme = text[pos : lastEnd]
kind = classify(lastFinal, lexeme)    # aquí se consulta la tabla de búsqueda
if kind not in {WHITESPACE, COMMENT}:
    tokens.add(kind, lexeme, line, col)
advanceLineCol(lexeme); pos = lastEnd

```

Pruebas del analizador léxico

Las pruebas se ejecutaron con el analizador desde la línea de comandos mediante `python main.py -text "..."`. La salida tiene tres partes: una línea con la clase de cada token en orden, una línea Clase: lexema por token y el total de tokens (Total of tokens: N). Los errores léxicos se imprimen al final con el formato Lexical error línea:columna: mensaje y no se cuentan como tokens. Los comentarios y los espacios se descartan, por lo que no aparecen en la salida. Las comillas dobles internas se escriben con `\` porque la cadena va entre comillas en la consola.

T1. Caso correcto: declaración, cadenas y operador &

Entrada con una declaración, dos llamadas con cadenas y el operador &:

```

unam.fi.compilers.g5.06>python main.py --text "int number; printf(\"Enter a
    number:\"); scanf(\"%d\", &number); "
keyword identifier punctuation identifier punctuation literal punctuation
punctuation identifier punctuation literal punctuation operator identifier
punctuation punctuation
Keyword: int
Identifier: number
Punctuation: ;
Identifier: printf
Punctuation: (
Literal: "Enter a number:"
Punctuation: )
Punctuation: ;
Identifier: scanf
Punctuation: (
Literal: "%d"
Punctuation: ,
Operator: &
Identifier: number
Punctuation: )
Punctuation: ;
Total of tokens: 16

```

Se observa que `int` sale como palabra reservada y `number` como identificador (regla 2). Con

printf funciona el máximo bocado (regla 1): aunque print es palabra reservada, el lexema más largo que termina en q_1 es printf, que no está en la tabla de búsqueda y se clasifica como ID. Las cadenas "Enter a number:" y "%d" terminan en q_6 y se emiten completas, con sus comillas. El & aislado termina en q_{14} y es un operador de un carácter, porque no le sigue otro &.

T2. Comentarios, decimales y palabras reservadas

Entrada con un decimal, un comentario de bloque, una cadena y un comentario de línea:

```
unam.fi.compilers.g5.06>python main.py --text "float x = 3.14; /* bloque */
    print(\"Hola\"); // fin"
keyword identifier operator constant punctuation keyword punctuation literal
    punctuation punctuation
Keyword: float
Identifier: x
Operator: =
Constant: 3.14
Punctuation: ;
Keyword: print
Punctuation: (
Literal: "Hola"
Punctuation: )
Punctuation: ;
Total of tokens: 10
```

El comentario de bloque (termina en q_{18}), el de línea (termina en q_9) y los espacios (q_{19}) se reconocen con las mismas reglas que los tokens, pero se descartan, así que no aparecen en la salida. El decimal 3.14 termina en q_4 y se emite como constante.

T3. Caso con error: constante de carácter mal formada

Entrada con una comilla simple que nunca se cierra:

```
unam.fi.compilers.g5.06>python main.py --text "int number = 'a; printf(\"Enter a
    number:\"); scanf(\"%d\", &number); "
keyword identifier operator
Keyword: int
Identifier: number
Operator: =
Total of tokens: 3
Lexical error 1:14: malformed character constant
```

El AFD lee 'a y llega a q_{23} , pero el siguiente carácter es ; y desde q_{23} solo hay transición con '. El escaneo se detiene sin haber pasado por un estado final, así que se reporta el error en la línea 1, columna 14 (la posición de la comilla), con el mensaje de la Tabla 8 para q_{21} , q_{22} y q_{23} . La

recuperación salta hasta la siguiente ' de la línea o, si no hay otra, hasta el final de la línea. Aquí no hay otra comilla simple, por lo que se descarta el resto de la línea y por eso solo se emiten los tres tokens anteriores al error.

T4. Entrada para el analizador sintáctico

Entrada con la declaración usada en la sección CFG:

```
unam.fi.compilers.g5.06>python main.py --text "int a = 10;"
keyword identifier operator constant punctuation
Keyword: int
Identifier: a
Operator: =
Constant: 10
Punctuation: ;
Total of tokens: 5
```

Los cinco tokens corresponden a INT, ID, ASSIGN, INT_CONST y SEMI, que son los terminales int, id, =, num y ; de la gramática G_1 .

CFG

A continuación se presenta la Gramática Libre de Contexto $G_0 = (N, T, P, S)$ para la declaración, con $N = \{\text{Decl, Type, Value}\}$, $T = \{\text{int, float, char, id, num, =, ;}\}$ y $S = \text{Decl}$.

1. $\text{Decl} \rightarrow \text{Type id ;}$
2. $\text{Decl} \rightarrow \text{Type id = Value ;}$
3. $\text{Type} \rightarrow \text{int} \mid \text{float} \mid \text{char}$
4. $\text{Value} \rightarrow \text{num} \mid \text{id}$

Se eligió una declaración de variable porque permite mostrar un caso de producciones con prefijo común con el fin de aplicar producciones por left factoring.

Se tomó la secuencia de tokens int id = num ; y se escribió una producción por cada forma de la declaración sin valor y con valor. Los terminales son tokens del siguiente catálogo:

Terminal	TokenType
int, float, char	INT, FLOAT, CHAR
id	ID
num	INT_CONST
=	ASSIGN
;	SEMI

Existe una consideración para la generación de LL(1). Las producciones 1 y 2 comparten el prefijo Type id. Después de leer int a, otro tipo de analizador no tendría esta consideración, por consecuencia, no sabría cuál aplicar, por lo tanto, en una tabla LL(1) ambas caerían en la celda [Decl, int].

Left factoring

Se factorizó el prefijo común de G_0 y se verificó que el resultado, G_1 , es LL(1). Así mismo, se buscó el prefijo común más largo de las dos producciones de Decl: $\alpha = \text{Type id}$. Lo que sigue en cada una es $\beta_1 = ;$ y $\beta_2 = = \text{Value} ;$. Se aplicó la regla vista en clase.

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \implies A \rightarrow \alpha A', \quad A' \rightarrow \beta_1 \mid \beta_2$$

1. Decl $\rightarrow$ Type id Decl'
2. Decl' $\rightarrow ; \mid = \text{Value} ;$
3. Type $\rightarrow \text{int} \mid \text{float} \mid \text{char}$
4. Value $\rightarrow \text{num} \mid \text{id}$

La gramática factorizada es $G_1 = (N', T, P', S)$, con $N' = \{\text{Decl}, \text{Decl}', \text{Type}, \text{Value}\}$, el mismo conjunto de terminales T de G_0 , el símbolo inicial $S = \text{Decl}$ y P' formado por las producciones anteriores. N' incluye el no terminal nuevo Decl' introducido por la factorización.

La gramática resultante no presenta left factoring y elimina el prefijo común de las producciones de Decl. Posteriormente se verificó mediante FIRST, FOLLOW y la tabla predictiva que no presenta conflictos para este ejemplo.

FIRST, FOLLOW, tabla LL(1) y traza.

En la siguiente tabla se muestra el resultado de las operaciones First y Follow para la gramática.

No terminal	FIRST	FOLLOW
Decl	{int, float, char}	{\$}
Decl'	{;, =}	{\$}
Type	{int, float, char}	{id}
Value	{num, id}	{;}

Tabla 9. Conjuntos FIRST y FOLLOW calculados para la gramática G_1 .

En la siguiente tabla se define la tabla LL(1).

	int	float	char	id	num	=	;	\$
Decl	Decl $\rightarrow$ Type id Decl'	Decl $\rightarrow$ Type id Decl'	Decl $\rightarrow$ Type id Decl'					
Decl'						Decl' $\rightarrow$ = Value ;	Decl' $\rightarrow$;	
Type	Type $\rightarrow$ int	Type $\rightarrow$ float	Type $\rightarrow$ char					
Value				Value $\rightarrow$ id	Value $\rightarrow$ num			

Tabla 10. Tabla de análisis sintáctico predictivo LL(1) para la gramática G_1 .

Cada celda tiene a lo más una producción, así que G_1 es LL(1). Traza de `int id = num ;`, con el tope de la pila a la derecha como en clase:

Paso	Pila	Entrada	Acción
1	\$ Decl	int id = num ; \$	Decl $\rightarrow$ Type id Decl'
2	\$ Decl' id Type	int id = num ; \$	Type $\rightarrow$ int
3	\$ Decl' id int	int id = num ; \$	match int
4	\$ Decl' id	id = num ; \$	match id
5	\$ Decl'	= num ; \$	Decl' $\rightarrow$ = Value ;
6	\$; Value =	= num ; \$	match =
7	\$; Value	num ; \$	Value $\rightarrow$ num
8	\$; num	num ; \$	match num
9	\$;	; \$	match ;
10	\$	\$	éxito

Tabla 11. Traza del análisis sintáctico predictivo para la cadena `int id = num ;`

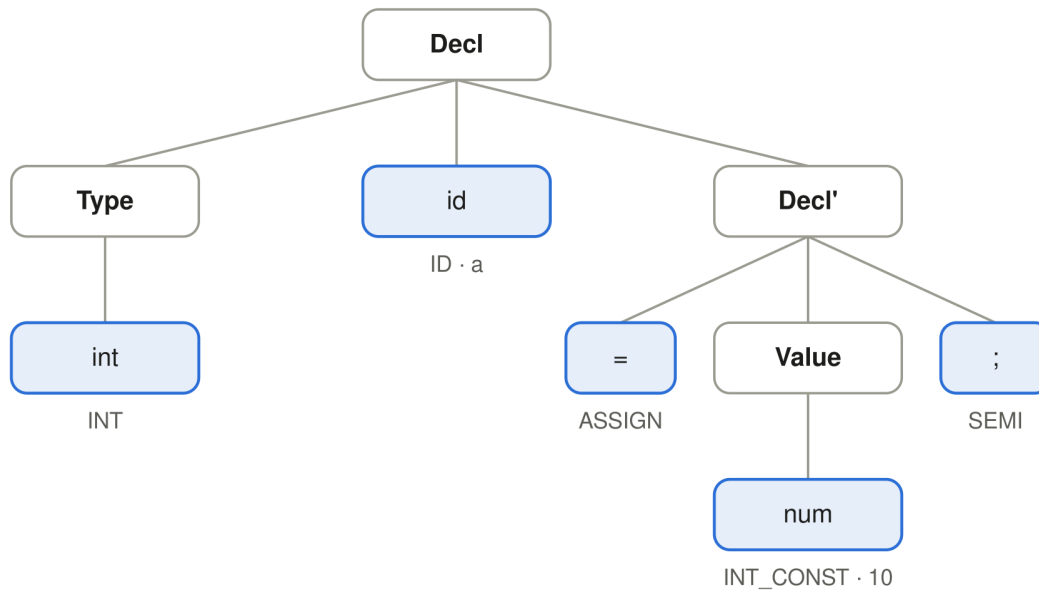


Figura 5: Árbol sintáctico de `int a = 10;` · gramática G_1

Las hojas del árbol son los tokens INT, ID, ASSIGN, INT_CONST y SEMI: exactamente la salida del lexer para la prueba T4 (sección Pruebas del analizador léxico).

Conclusiones

Este proyecto ofreció un analizador léxico construido desde la definición de las gramáticas hasta su implementación. Un listado de 45 tokens agrupados en palabras reservadas, identificadores, constantes, literales, operadores, puntuación y caracteres especiales fue el primer paso de diseño; también se diseñaron las expresiones regulares para estos tokens. Las expresiones regulares se tradujeron a un AFND de nueve sub-autómatas unidos por transiciones ϵ . Este autómata se convirtió en un autómata determinista de 25 estados mediante las técnicas de ϵ -cerradura y *mover*.

La regla del máximo bocado, con retroceso al último estado final visitado, resuelve la mayoría de las ambigüedades. Las palabras reservadas y los booleanos no necesitaron estados propios, porque se reconocen como identificadores y se clasifican con una tabla de búsqueda. Tratar + y - como operadores independientes evita ambigüedades en expresiones como $a-1$. El único no determinismo real del lenguaje, el carácter /, con facultad de iniciar un comentario o ser un operador, quedó resuelto en el estado q_8 . La excepción del comentario de bloque sin cerrar evita que el escáner retroceda y reporte un / en lugar del error real.

El manejo de errores léxicos con recuperación en modo pánico permite que el analizador no se detenga ante el primer problema. Reporta línea y columna, y continúa con el resto de la entrada. La arquitectura en etapas (lectura, preprocesamiento, escaneo, clasificación y salida), con tabla de símbolos y manejador de errores, separa responsabilidades y facilita modificar cada parte por separado.

Por último, la CFG de la declaración de variables, con su *left factoring*, los conjuntos FIRST y FOLLOW y la tabla LL(1), mostró cómo el analizador léxico alimenta al sintáctico. Las hojas del árbol de `int a = 10;` son exactamente los tokens que produce el lexer, lo que confirma que ambas fases quedaron bien conectadas. Como trabajo futuro, el lexer podría ampliarse con notación científica, operadores compuestos (`++`, `+=`) o comentarios anidados, y el tipo de cada identificador podría completarse en la tabla de símbolos durante el análisis semántico.